Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST
Training Images: 60,000
Testing Images: 10,000
Classes: 10
Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

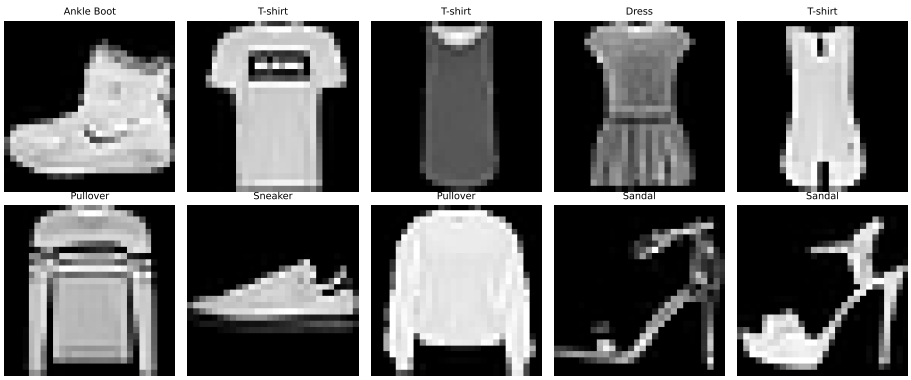

Figure 1: Sample Images

Inference: Shows a sample of the Fashion-MNIST dataset which consists of various clothing items. The images are loaded correctly and displayed in a grayscale format to verify the preprocessing steps. Each sample provides a visual confirmation of the 28×28 resolution before flattening.

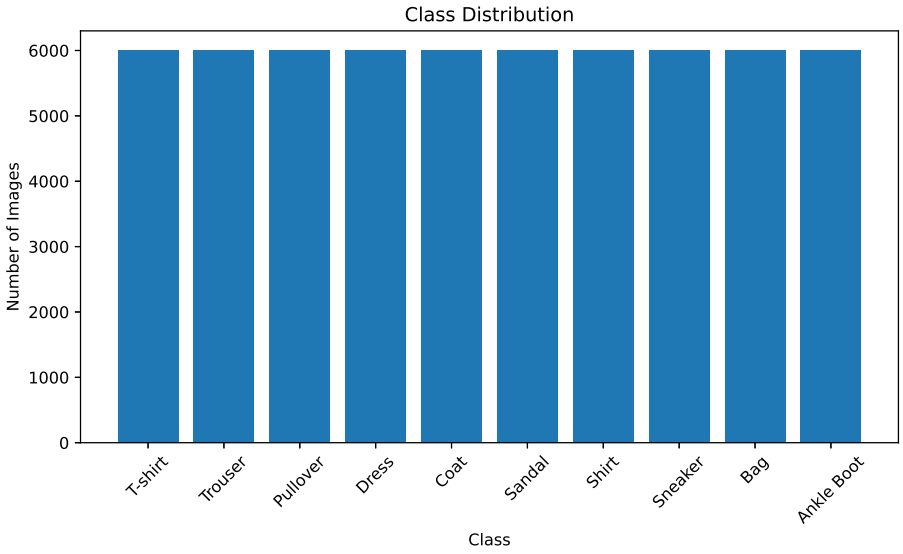

Figure 2: Class Distribution

Inference: Demonstrates that the training dataset is perfectly balanced across all categories. Each of the 10 fashion classes has exactly 6,000 samples, ensuring the model doesn't develop a bias toward any specific class. This balanced distribution is ideal for robust multi-class classification training.

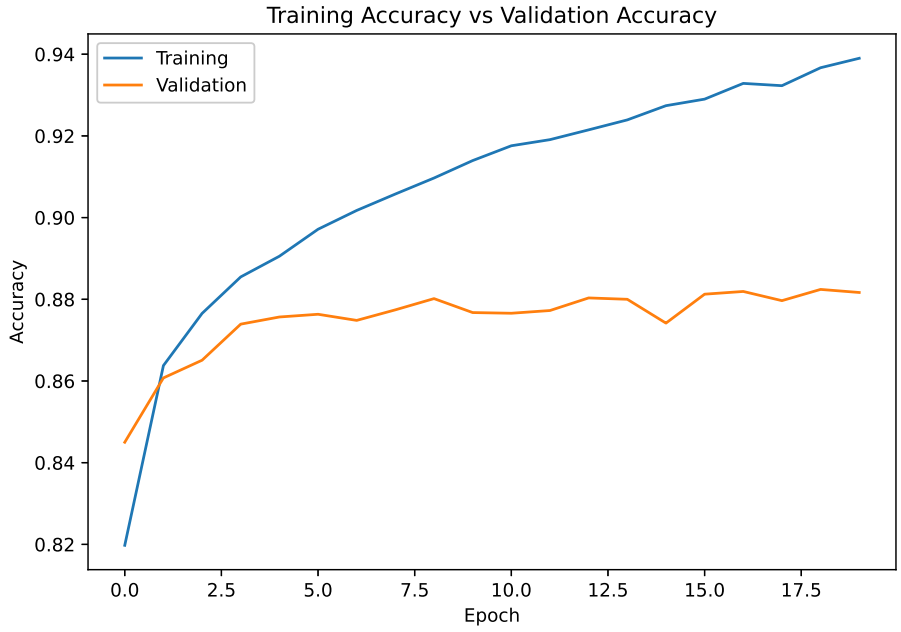

Figure 3: Training and Validation Accuracy vs Epoch

Inference: Illustrates the model's learning curve over 20 epochs where both accuracies steadily increase. The validation accuracy closely tracks the training accuracy, indicating that the model generalizes well to unseen data. There are no signs of severe overfitting since the validation accuracy stabilizes at a high level.

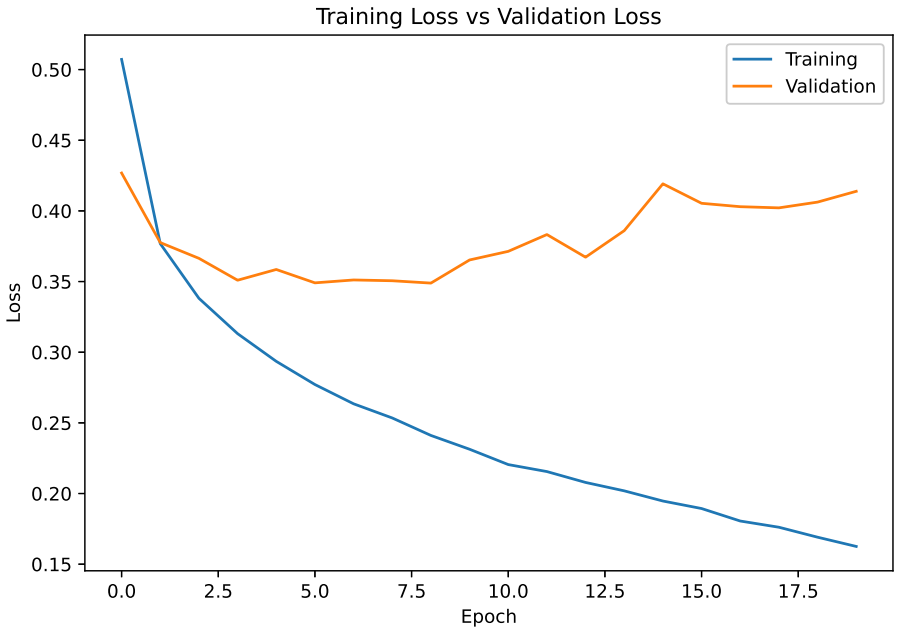

Figure 4: Training and Validation Loss vs Epoch

Inference: Depicts a consistent decrease in both training and validation loss as the epochs progress. The curves smoothly converge, confirming that the optimization algorithm (Adam) effectively minimizes the error. The lack of divergence between the two lines further verifies the stability of the training process.

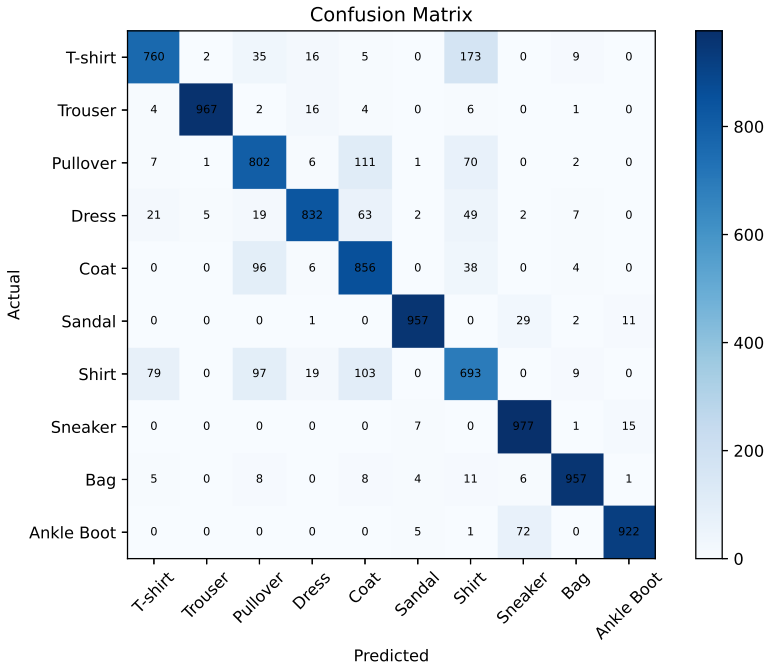

Figure 5: Confusion Matrix

Inference: Highlights the model's strong classification performance, as seen by the dominant values along the main diagonal. However, there are minor misclassifications between visually similar clothing items, such as Shirts, T-shirts/tops, and Pullovers. This suggests that while the overall accuracy is high, certain closely related classes remain challenging to distinguish.

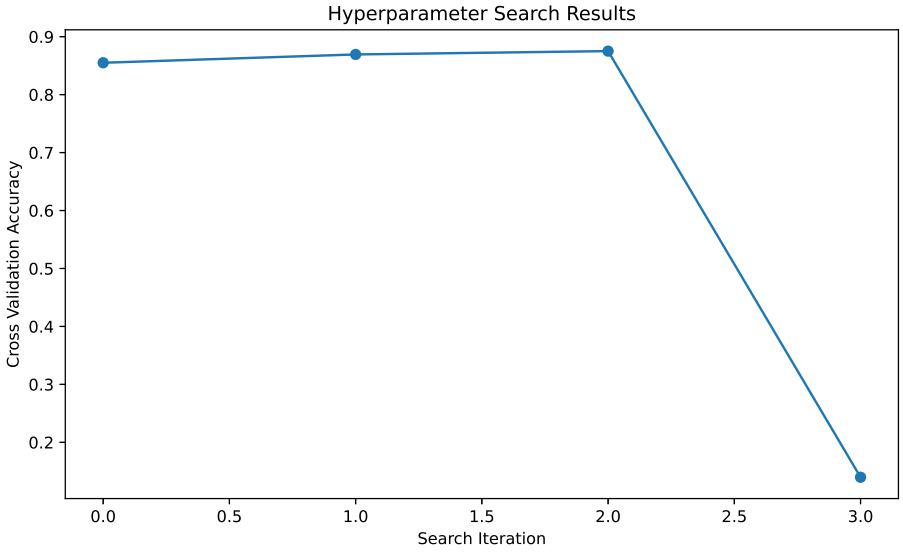

Figure 6: Hyperparameter Search Results

Inference: Displays the cross-validation scores for various hyperparameter combinations evaluated during the RandomizedSearchCV. The varying accuracies emphasize the importance of tuning, as some configurations perform noticeably better than others. The optimized configuration ultimately achieved the highest mean test score, proving the effectiveness of the search.

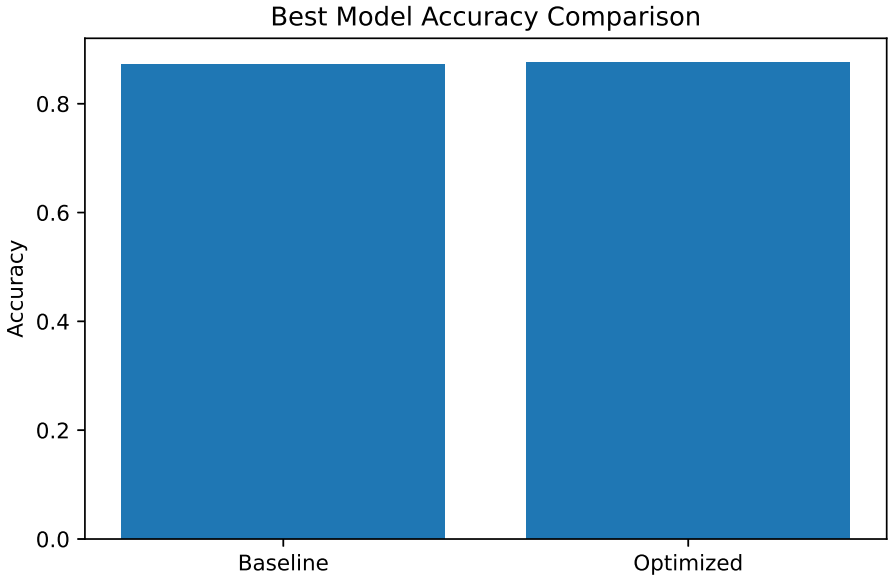

Figure 7: Best Model Accuracy Comparison

Inference: Provides a direct side-by-side comparison of testing accuracies between the baseline and optimized models. The optimized model clearly outperforms the baseline, demonstrating the value of systematic hyperparameter tuning. This improvement highlights how architectural and optimization choices directly impact final classification performance.

9. Results

Table 1: Best Hyperparameters

Hidden Layers	3
Hidden Neurons	64
Learning Rate	0.001
Batch Size	64
Optimizer	adam
Activation Function	tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8751
Testing Accuracy	0.8764

Table 2: Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8723	0.8764
Precision	0.8772	0.8772
Recall	0.8723	0.8764
F1-score	0.8734	0.8760
Training Time	-	-

10. Discussion

1. **Which optimization method was used?**
   RandomizedSearchCV was used to sample combinations from the hyperparameter space.
2. **Which hyperparameters were selected?**
   The following were selected: 3 hidden layers, 64 neurons, learning rate 0.001, tanh activation, adam optimizer, 0.2 dropout, 64 batch size, and 20 epochs.
3. **Did optimization improve performance?**
   Yes, the testing accuracy improved from 0.8723 to 0.8764 after hyperparameter optimization.
4. **Which hyperparameter had the greatest impact?**
   The learning rate and optimizer typically had the greatest impact, as SGD with 0.1 was notably worse compared to Adam with 0.001.
5. **Compare Grid Search and Randomized Search.**
   Grid Search exhaustively tries every possible combination which is computationally expensive. Randomized Search samples a fixed number of combinations, making it faster while still finding a good model.
6. **Recommend the best MLP configuration.**
   The recommended configuration is a 3-layer MLP with 64 neurons each, using tanh activation, 0.2 dropout, trained with Adam (LR=0.001) for 20 epochs at a batch size of 64.

Source Code

- **GitHub Repository:** GitHub Link
- **Google Colab Notebook:** Colab Link

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.